Remote invocation
(aka intersystem communication)

Agenda

From local to distributed programming

Request-reply protocols and HTTP

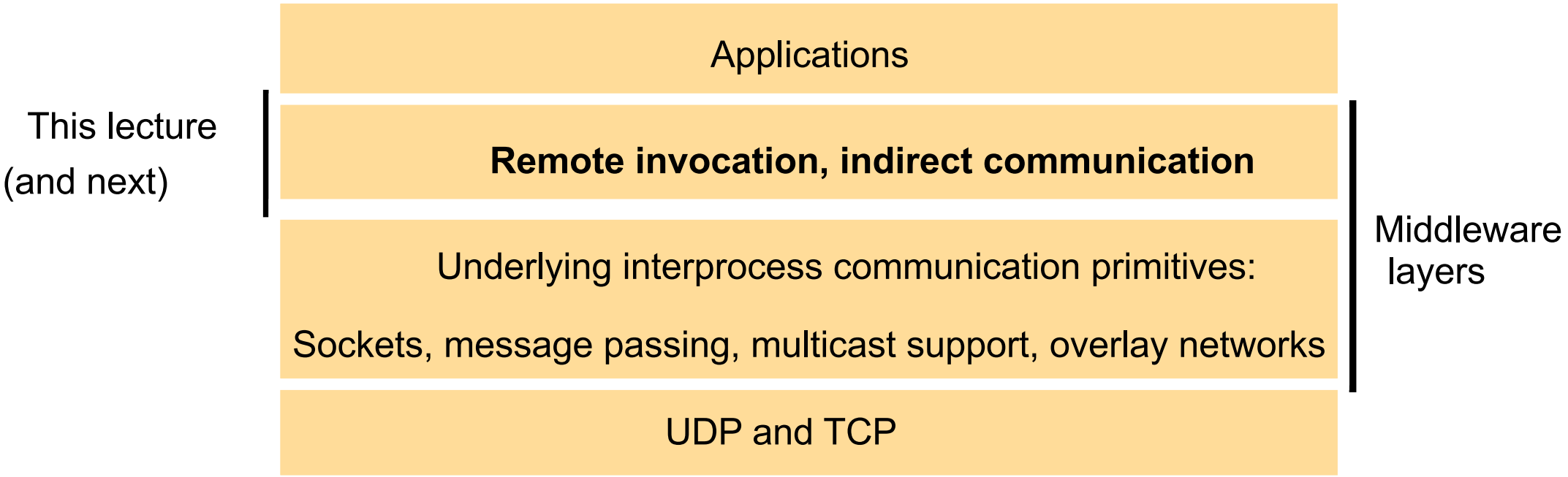
Remote Procedure Call (RPC) and Remote Method Invocation (RMI)

Call semantics

Interface Definition Languages (IDL) and interface evolution

API styles, data serialization, Protocol Buffers and gRPC

Middleware layers



Programming locally vs programming remotely

Multiple processors

No shared memory

Parallel execution

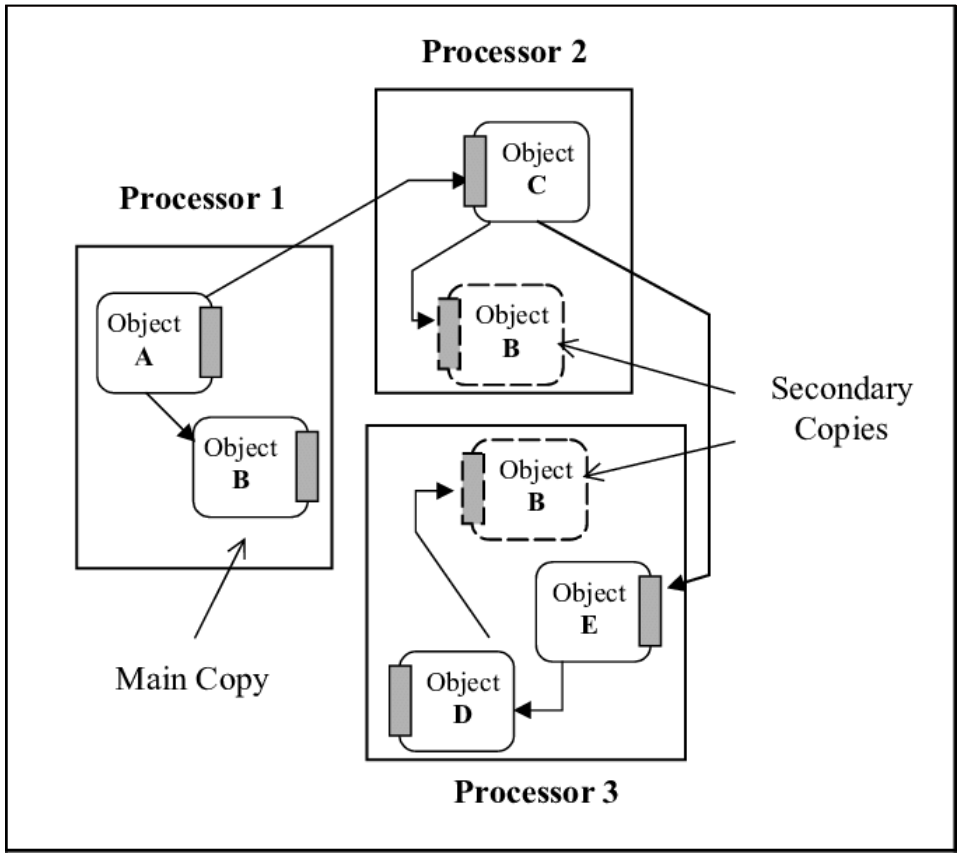
Remote variables?

Remote procedures?

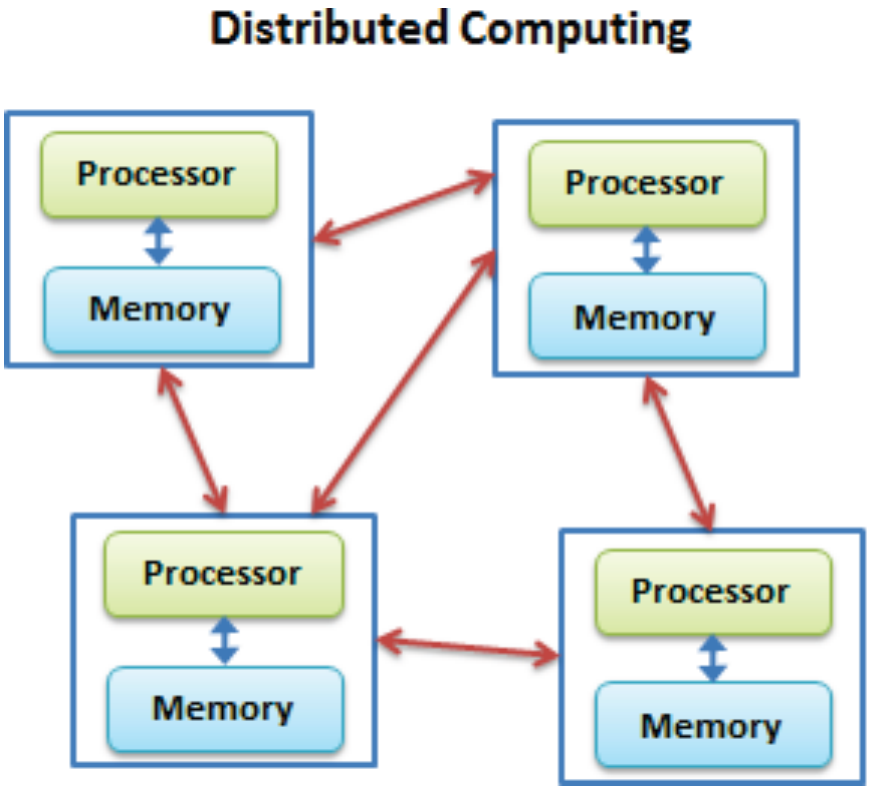
Remote invocation types?

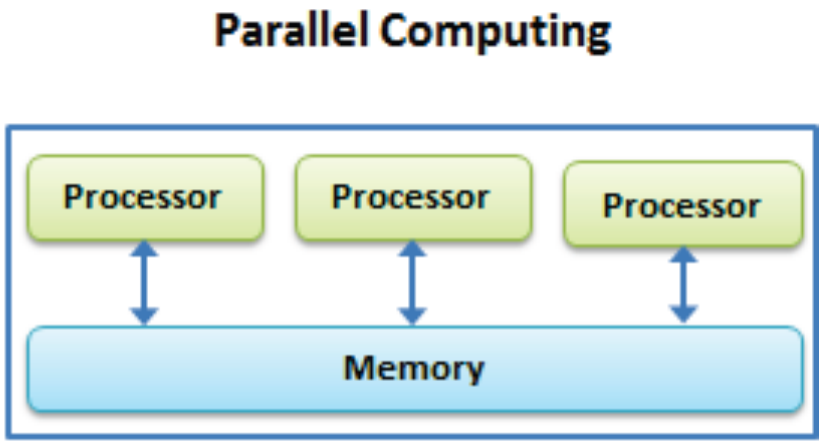
Remote data? Partitioning?

Mobile code, data, objects?



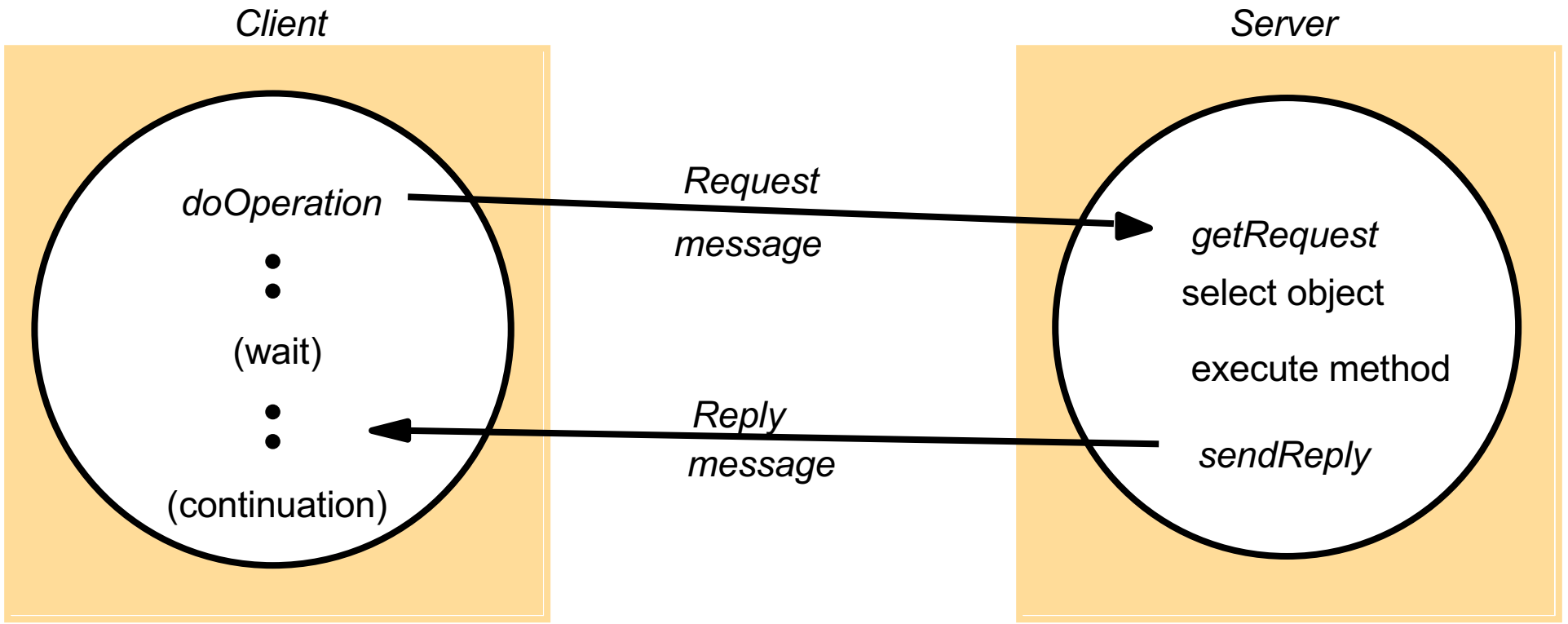
Parallel vs distributed computing





Multiprocessor with shared memory (multicore)

Request-reply communication



A request-reply protocol matches requests to replies.
It may be designed to provide certain delivery guarantees.
If UDP datagrams are used, the delivery guarantees must be provided by the request-reply protocol itself, which may use the server reply message as an acknowledgement of the client request message.

Operations of the request-reply protocol

public byte[] ***doOperation*** *(RemoteRef s, int operationId, byte[] arguments)*

sends a request message to the remote server and returns the reply.

The arguments specify the remote server, the operation to be invoked and the arguments of that operation.

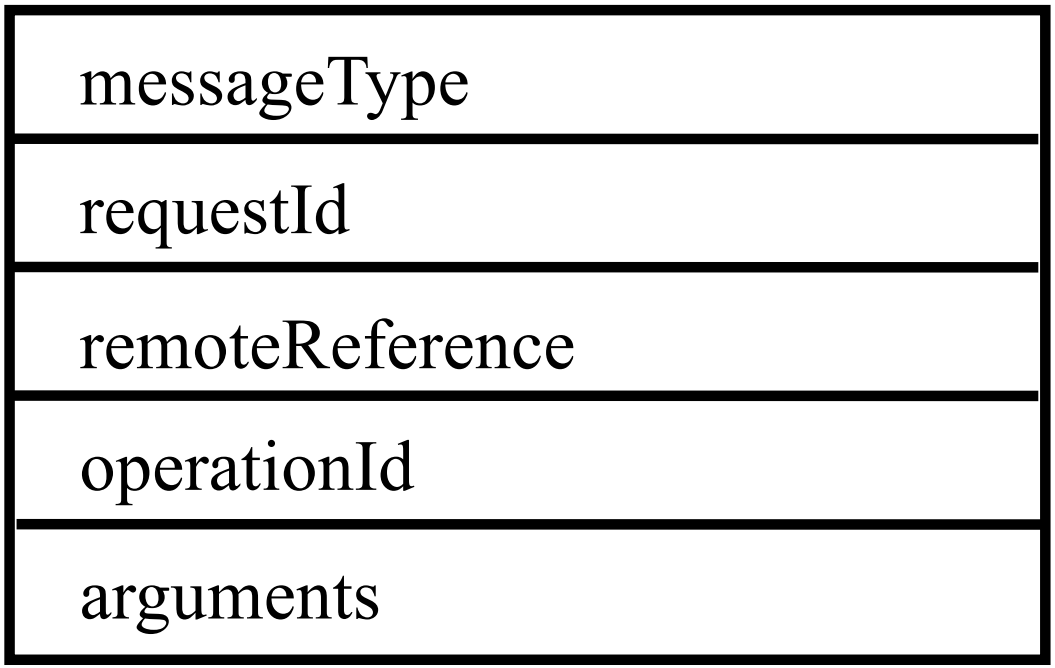
public byte[] ***getRequest*** *();*

acquires a client request via the server port.

public void ***sendReply*** *(byte[] reply, InetAddress clientHost, int clientPort);*

sends the reply message reply to the client at its Internet address and port.

Request-reply message structure



Request-reply protocols: variants

Name	*Messages sent by*		
	Client	*Server*	*Client*
R	*Request*		
RR	*Request*	*Reply*	
RRA	*Request*	*Reply*	*Acknowledge reply*

The protocols produce differing behaviours in presence of communication failures and are used for implementing various types of request behaviour:

- the request (R) protocol;
- the request-reply (RR) protocol;
- the request-reply-acknowledge reply (RRA) protocol.

HTTP as a request-reply protocol

HTTP/1.1 and HTTP/2 run over TCP; HTTP/3 runs over QUIC (UDP)

In the original version of the protocol, each client-server interaction consisted of the following steps:

1. The client requests and then the server accepts a connection at the default server port or at a port specified in the URL.
2. The client sends a request message to the server.
3. The server sends a reply message to the client.
4. The connection is closed

Later versions keep connections open (HTTP/1.1) and multiplex many requests on one connection (HTTP/2, HTTP/3).

HTTP methods: GET

- Each client request specifies the name of a **method** to be applied to a resource at the server and the URL of that resource.
- The reply reports on the status of the request. Requests and replies may also contain resource data, the contents of a form or the output of a program resource run on the web server.

GET: Requests the resource whose URL is given as its argument.

If the URL refers to data, then the web server replies by returning the data identified by that URL.

If the URL refers to a program, then the web server runs the program and returns its output to the client.

With GET, all the information for the request is provided in the URL

HTTP methods: safety and idempotence

GET: retrieves the resource identified by the URL (data, or the output of a program).

HEAD: like GET, but returns only metadata (last modification time, type, size).

POST: sends data in the body to the resource for processing; may change server state.

PUT: stores the supplied data under the given URL, creating or replacing the resource.

PATCH: applies a partial modification to the resource.

DELETE: deletes the resource; the server may refuse.

OPTIONS: lists the methods allowed on the URL and its requirements.

TRACE: echoes the request back, for diagnostics.

Safe (no state change): GET, HEAD, OPTIONS, TRACE.

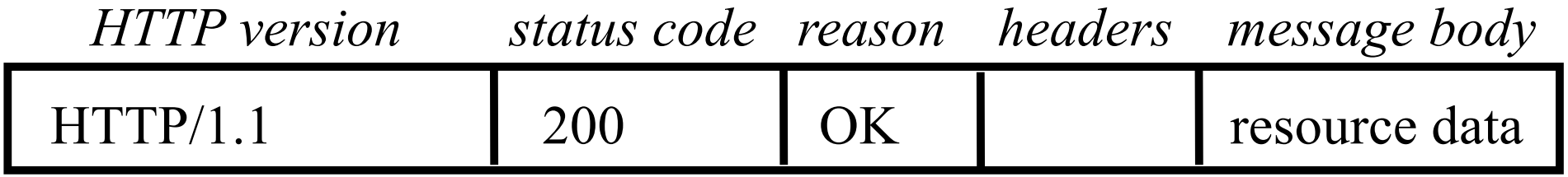
Idempotent (repeating has the same effect as doing it once): the safe methods plus PUT and DELETE. POST and PATCH are not necessarily idempotent, which matters when a client retries a request.

HTTP request message (method GET) to a web server

method	*URL or pathname*	*HTTP version*	*headers*	*message body*
GET	http://www.dcs.qmul.ac.uk/index.html	HTTP/ 1.1		

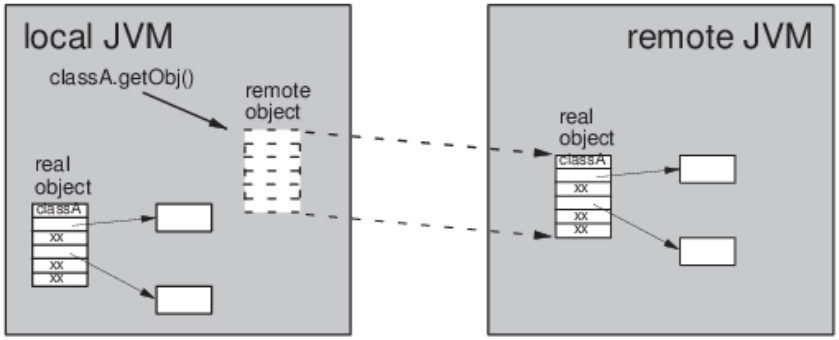
The Request message specifies the name of a method, the URL of a resource, the protocol version, some headers and an optional message body.
We show the contents of an HTTP Request message whose method is GET

HTTP *Reply* message (from the web server)



A Reply message specifies the protocol version, a status code and 'reason', some headers and an optional message body.
The status code and reason provide a report on the server's success or otherwise in carrying out the request

RPC and RMI



There are two main remote invocation techniques for communication in distributed systems:

- The remote procedure call (RPC) approach extends the abstraction of the procedure call to distributed environments, allowing a calling process to call a procedure in a remote node as if it were local
- Remote method invocation (RMI) is similar to RPC but for **distributed objects**, with added benefits in terms of using object-oriented programming concepts: extending the concept of an object reference to distributed environments, and allowing the use of object references as parameters in remote invocations.

From RR to RPC

- Request–reply exposes message exchange and communication details.
- RPC hides communication, improving access transparency.
- Yet latency and partial failures prevent complete transparency.

Implementation issues of remote calls

Transparency property

- Syntactic transparency
- Semantic transparency

Similarity between local and remote procedure calls

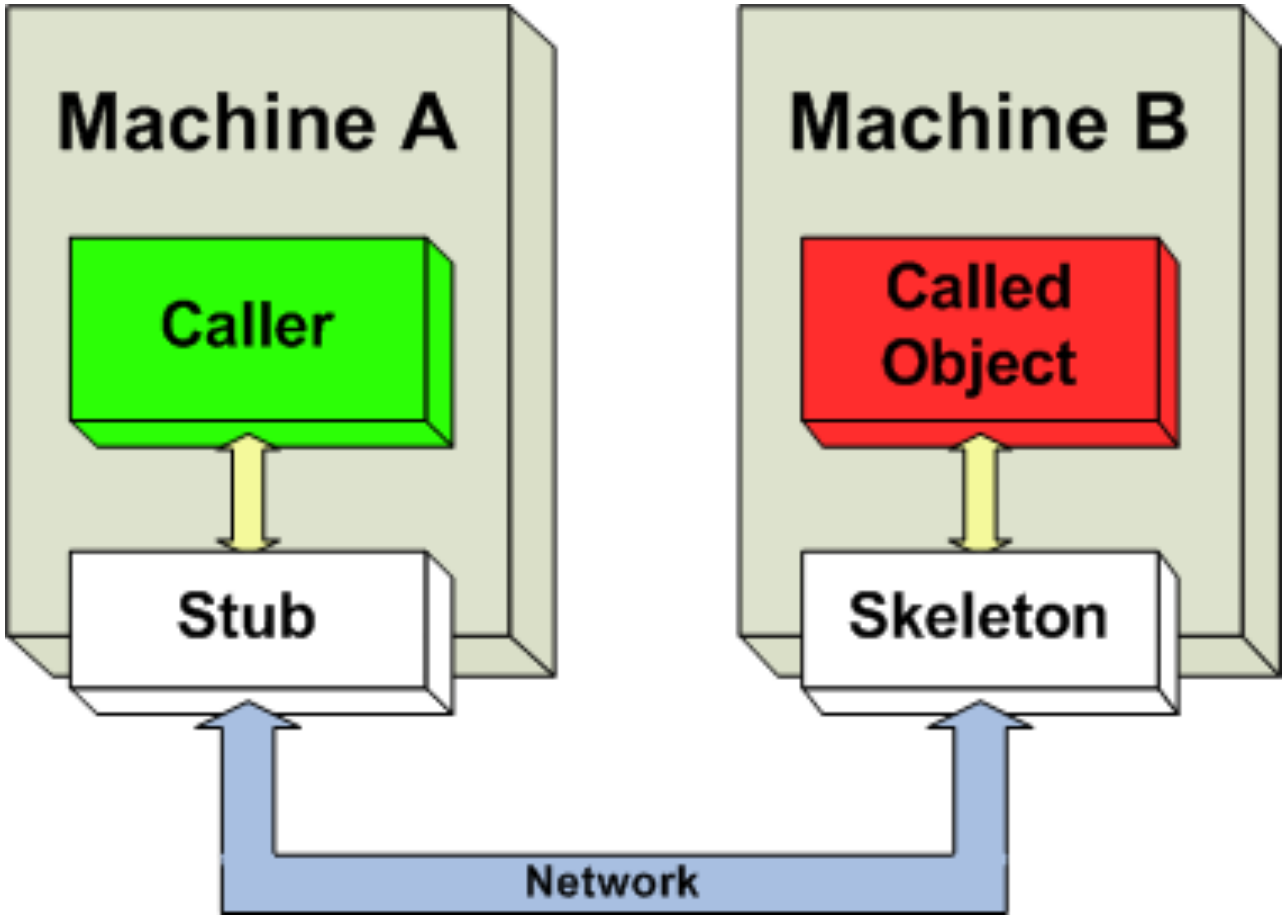
- Caller capable of passing arguments (automatic marshalling)
- Caller suspended till a return from a function
- Callee capable of returning a value to caller

Difference between local and remote calls

- No call by reference for local data: arguments are copied (RMI can pass remote object references)
- Error handling required for communication (Ex. RemoteException in Java)
- Performance much slower than local calls

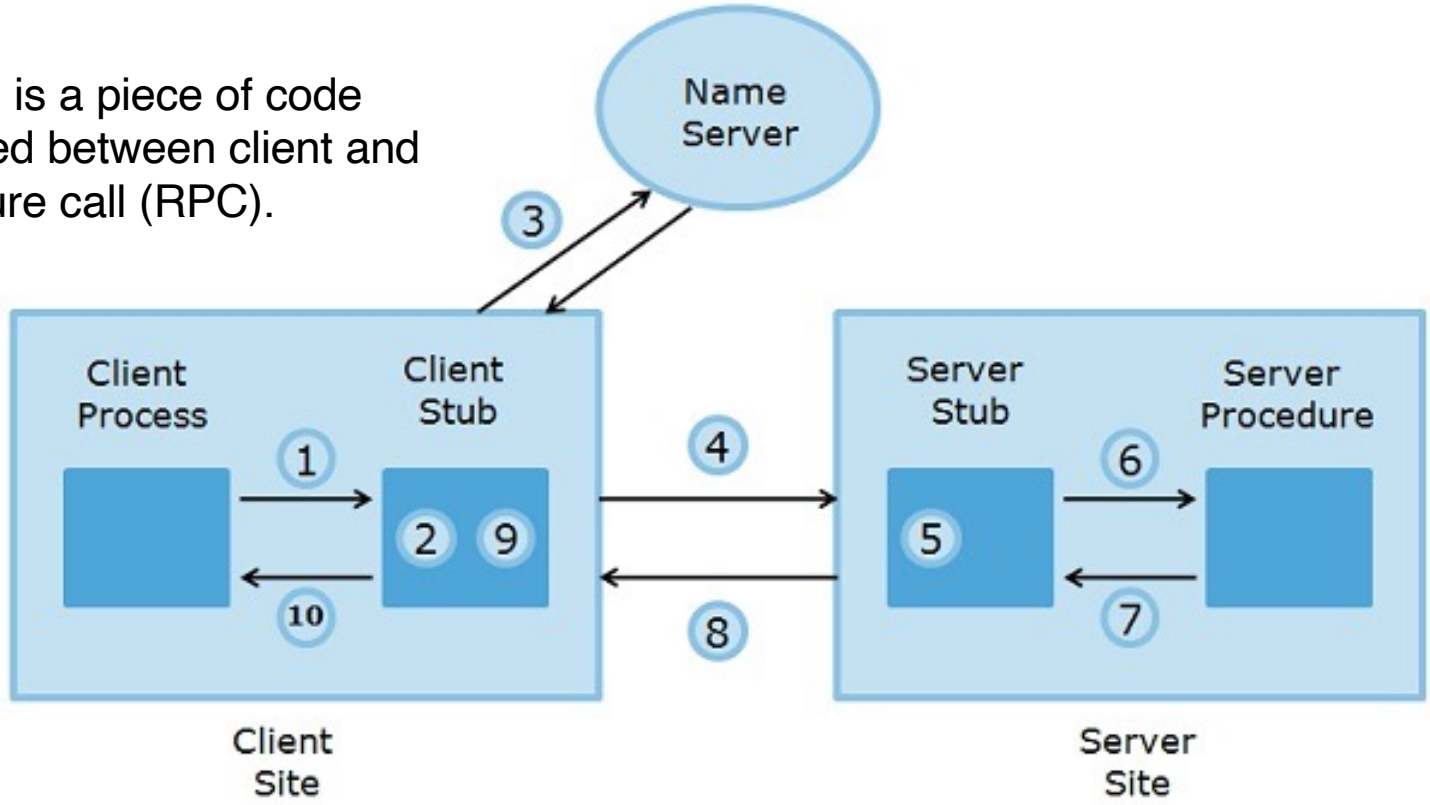
Implementing a RPC

Only the stubs, which are generated by an IDL compiler (or dynamically at run time), know that the call is remote
For the programmer (imagine launching her main program on machine A calling a remote object on B) the remote machine is «transparent»



RPC

A **stub** in distributed computing is a piece of code that converts parameters passed between client and server during a remote procedure call (RPC).



The client and server use different address spaces, so parameters used in a function (procedure) call have to be converted, otherwise the values of those parameters could not be used, because pointers to parameters in one computer's memory would point to different data on the other computer.
The client and server may also use different data representations, even for simple parameters (e.g., big-endian versus little-endian for integers).
Stubs perform the conversion of the parameters, so a remote procedure call looks like a local function call to the caller.

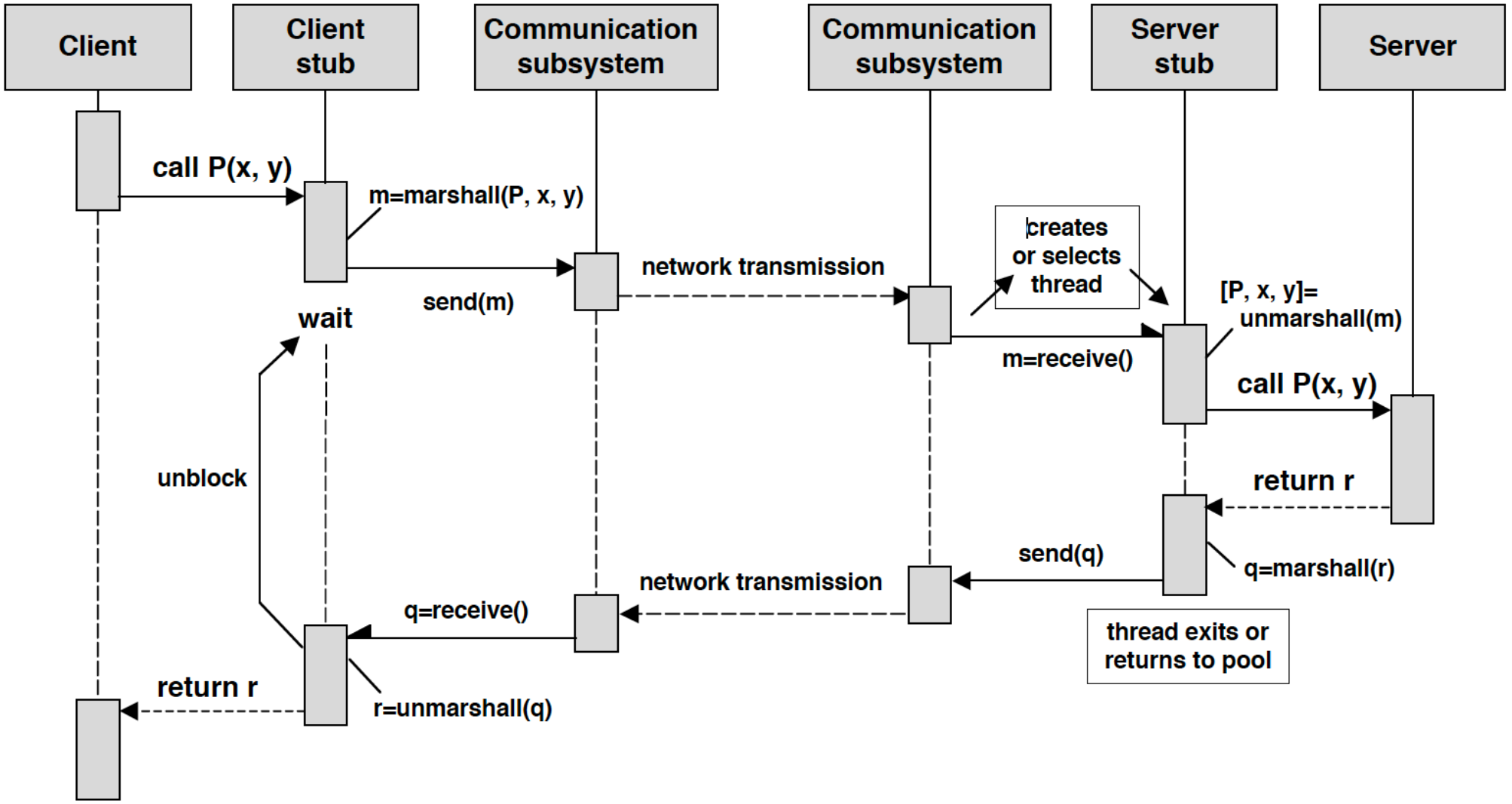
Marshalling and external data representation

- **Marshalling**: assembling a collection of data items into a form suitable for transmission; **unmarshalling**: rebuilding equivalent data at the destination.
- Why it is needed: byte order (big- vs little-endian), character encodings, structure layout and padding differ between machines; pointers mean nothing in another address space.
- Two strategies: convert to an agreed **external data representation**, or send in the sender's format plus an indication of it (receiver makes right).
- Binary formats (CORBA CDR, Java serialization, Protobuf) are compact; textual ones (XML, JSON) are readable but larger and slower to parse.
- Stubs do this automatically, from the IDL.

Bytes	4 bytes	Notes
0–3	`5`	length of string
4–7	`"Smit"`	"Smith"
8–11	`"h___"`	
12–15	`6`	length of string
16–19	`"Lond"`	"London"
20–23	`"on__"`	
24–27	`1984`	unsigned long

CORBA CDR message for `Person{'Smith', 'London', 1984}`*. Strings are padded to 4 bytes; types are not sent, because both sides know them from the IDL.*

RPC: overall flow of control



Stub and skeleton in Java RMI (classic design)

Stub: This is an object that acts as an entry point for the client object to route any outgoing requests. It exists on the client JVM and represents the handle to the remote object. If any object invokes a method on the stub object, the stub establishes RMI by following these steps:

1. It initiates a connection to the remote machine JVM.
2. It marshals (write and transmit) the parameters passed to it to the remote JVM.
3. It waits for a response from the remote object and unmarshals (read) the returned value or exception, then it responds to the caller with that value or exception.

Skeleton: This is an object that behaves like a gateway on the server side. It acts as a remote object with which the client objects interact through the stub. This means that any requests coming from the remote client are routed through it. If the skeleton receives a request, it establishes RMI through these steps:

1. It reads the parameter sent to the remote method.
2. It invokes the actual remote object method.
3. It marshals (writes and transmits) the result back to the caller (stub).

Remote object references

32 bits	32 bits	32 bits	32 bits	
Internet address	**port number**	**time**	**object number**	**interface of remote object**

- A **remote object reference** identifies a remote object throughout the distributed system: a client must hold one to invoke the object.
- It must be **unique in space and time**: if the reference of a deleted object were reused, a client could invoke the wrong object.
- Address and port identify the server process; creation time and object number keep the reference unique even if a new process later reuses the same port.
- This form ties the object to one process: if objects can migrate, a location service maps references to current addresses.
- References can be passed as arguments and results: in Java RMI the stub, which contains the reference, is serialized in place of the remote object.

Design issues for RPC

Three issues are important:

1. the style of programming promoted by RPC –that is, programming with interfaces;
2. the call semantics associated with RPC;
3. the issue of transparency and how it relates to remote procedure calls.

Synchronous vs asynchronous invocation

Synchronous (blocking)

- The caller is suspended until the reply or an exception arrives.
- Same programming model as a local call: easy to reason about.
- The caller idles for the whole round trip.
- Examples: Java RMI, a basic gRPC stub call.

Asynchronous (non-blocking)

- The caller continues; the result arrives later via a future/promise or a callback.
- Overlaps communication with computation; many calls in flight.
- Harder error handling, ordering and cancellation.

```
reply  = stub.Greet(req)          # synchronous: blocks until the reply
future = stub.Greet.future(req)   # asynchronous: returns at once
do_other_work()
reply  = future.result()          # blocks only if the reply is not there yet
```

One-way (fire-and-forget) calls have no reply at all: maybe semantics, e.g. CORBA oneway operations.

Call semantics

Fault tolerance measures			Call semantics
Retransmit request message	Duplicate filtering	Re-execute procedure or retransmit reply	
No	Not applicable	Not applicable	**Maybe**
Yes	No	Re-execute procedure	**At-least-once**
Yes	Yes	Retransmit reply	**At-most-once**

- **Maybe:** the procedure is executed once or not at all; the caller cannot tell which. Exposed to omission and crash failures.
- **At-least-once:** a result means the procedure ran one or more times; otherwise an exception. Safe only for idempotent operations (e.g. Sun RPC).
- **At-most-once:** a result means it ran exactly once; an exception means once or not at all. The server keeps a history of replies (e.g. Java RMI, CORBA).

Exactly-once cannot be guaranteed if the server may crash: this is one reason why RPC can never be fully transparent.

Timeouts, retries and idempotency

- Every remote call needs a **timeout**. gRPC uses **deadlines**, which propagate to downstream calls; expiry gives DEADLINE_EXCEEDED.
- A timeout does not tell whether the operation ran: the request, the execution or the reply may have been lost.
- Retry only what is safe to repeat: idempotent operations, or calls protected by duplicate filtering.
- Back off exponentially, with random jitter, so that many clients do not retry at the same moment.
- **Idempotency keys**: the client sends a unique key with each logical request; the server stores the result under that key and returns it on a retry.

Request-reply protocol	Today's practice
Timeout in doOperation	Call timeout / deadline
Retransmit the request	Retry with exponential backoff and jitter
requestId + reply history on the server	Idempotency key + stored response

Interface definition language (IDL)

- An RPC mechanism integrated with a programming language has to include a notation for defining interfaces, allowing input and output parameters to be mapped onto the language's normal use of parameters.
- This approach is useful when all the parts of a distributed application can be written in the same language. It is also convenient because it allows the programmer to use a single language for local and remote invocation.
- However, many existing useful services are written in different languages. It would be beneficial to allow programs written in a variety of languages, to access them remotely.
- Interface definition languages (IDLs) are designed to allow procedures implemented in different languages to invoke one another: an IDL provides a notation for defining interfaces in which each of the parameters of an operation may be described as for input or output in addition to having its type specified

IDL: examples

- AIDL: Java-based, for Android; supports local and remote procedure calls, can be accessed from native code through the Java Native Interface (JNI)
- Apache Avro IDL: schema language for Avro data and RPC
- Apache Thrift: from Apache, originally developed by Facebook
- AsyncAPI: a specification for event-driven (message-based) APIs
- Data Distribution Service (DDS): its IDL was identical to OMG IDL until version 3.5, then evolved independently
- GraphQL SDL: the schema definition language of GraphQL APIs
- Microsoft Interface Definition Language (MIDL): derived from DCE/RPC IDL, extended for COM and DCOM
- OMG IDL: the IDL of CORBA; also used by the W3C to specify the DOM
- OpenAPI Specification: a standard for REST interfaces
- Protocol Buffers: Google's IDL for gRPC
- Universal Network Objects (UNO): the OpenOffice/LibreOffice component model
- Web IDL: describes interfaces implemented in web browsers
- Web Services Description Language (WSDL): for SOAP web services

A simple CORBA IDL example

```
// In file Person.idl
struct Person {
        string name;
        string place;
        long year;
} ;
interface PersonList {
        readonly attribute string listname;
        void addPerson(in Person p) ;
        void getPerson(in string name, out Person p);
        long number();
};
```

Simple API:

The interface PersonList specifies the methods available in a remote object that implements that interface.

Methods:

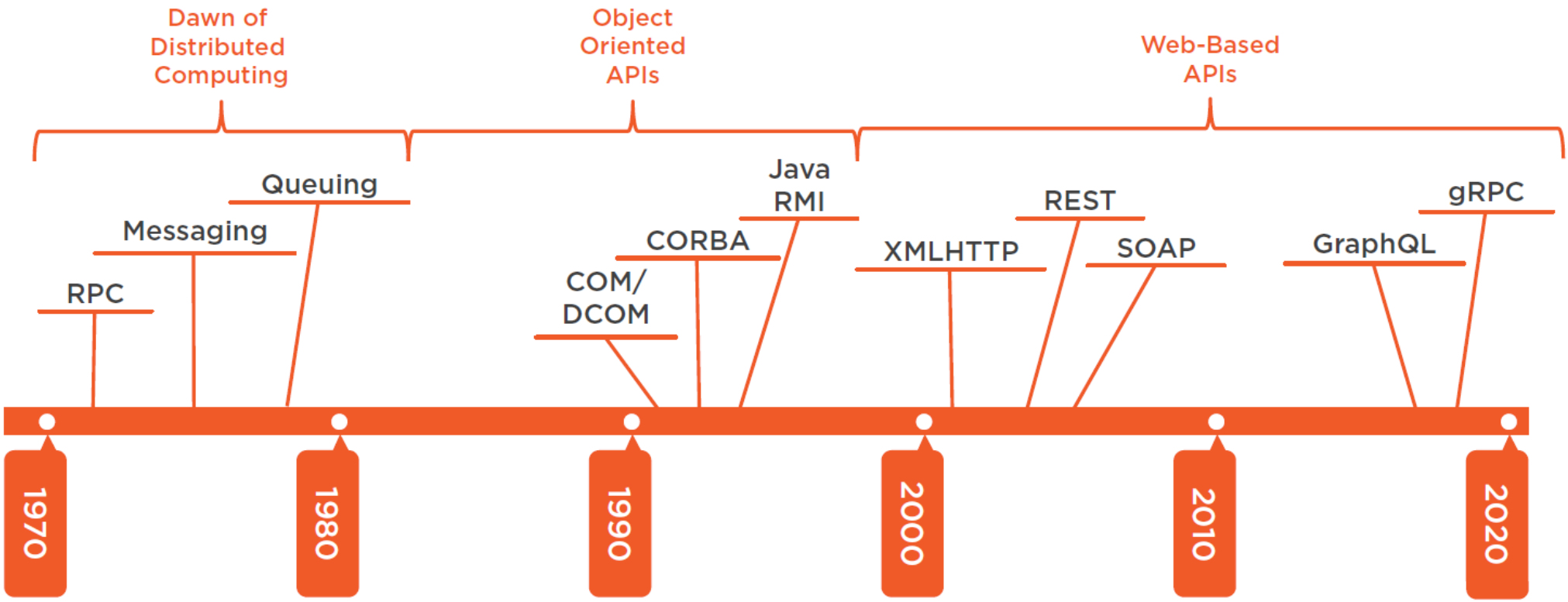
- the method *addPerson* specifies its argument as input (sent by the client)
- the method *getPerson*, that retrieves an instance of Person by name, specifies its second argument as output (computed by the server)

Interface evolution

- Once an interface is published, clients and servers are deployed and upgraded independently: a change must not break existing callers.
- Classic approaches: a new interface that extends the old one, or a versioning policy (incompatible changes only in a new major version).
- Schema-based IDLs build compatibility into the encoding. In Protocol Buffers **field numbers**, not names, travel on the wire:
  - adding a field is safe: old readers skip it, new readers see a default value when it is missing;
  - a number must never be reused or change type: removed numbers are marked reserved.
- Thrift (field ids) and Avro (writer/reader schema resolution) follow the same idea; REST and GraphQL APIs evolve by adding fields and deprecating old ones.

```
message Person {
  string name = 1;
  reserved 2;           // removed in v2
  int32  year = 3;
  string email = 4;     // added in v2
}
```

The History of Distributed APIs



Data serialization

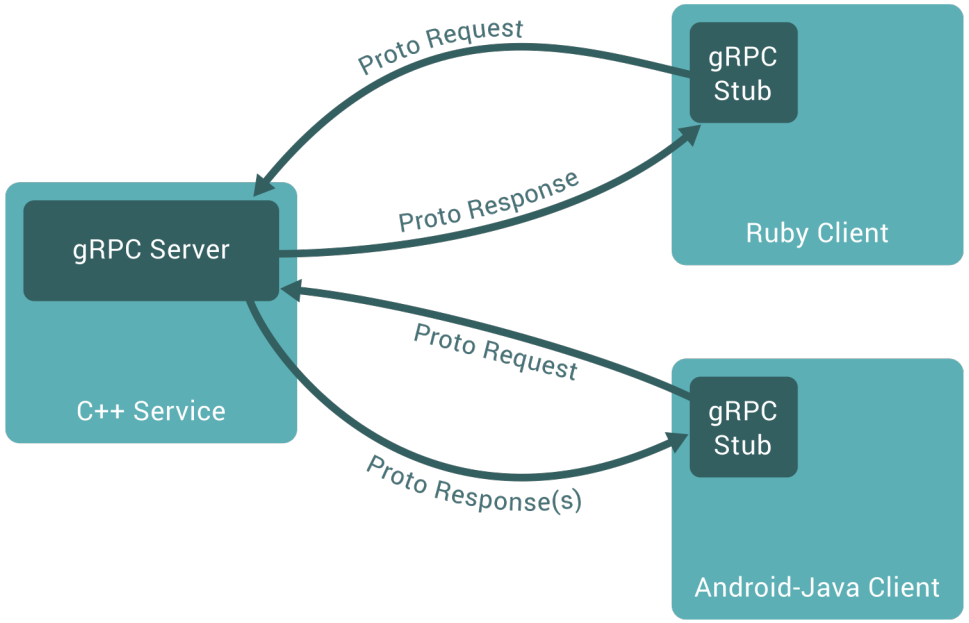
- Serialization converts structured data into transferable byte sequences. Common formats include XML, JSON, BSON, YAML, and MessagePack.
- Schema-based formats include Protobuf, Thrift, and Avro.
- Format choice depends on readability, size, latency, and tooling. Schema evolution and language interoperability are also important.
- JSON dominates web APIs because humans can inspect it easily.
- Binary formats usually provide compact encoding and faster processing.
- Protobuf, Thrift, and Avro support cross-language data exchange.

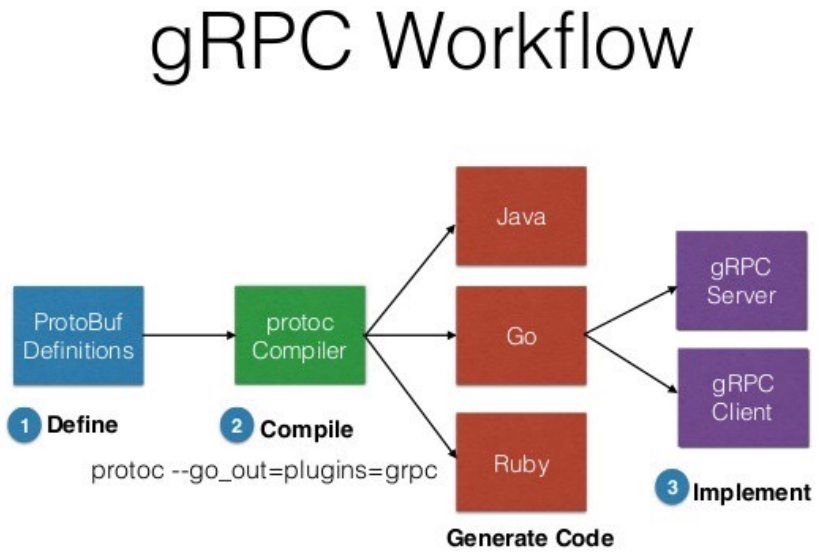
API styles compared

Style	Contract (IDL)	Encoding	Transport	Coupling and evolution
SOAP	WSDL + XML Schema	XML	HTTP (also others)	Tight: stubs generated from WSDL; new versions usually mean new namespaces or endpoints
REST	OpenAPI (optional)	JSON (any media type)	HTTP	Loose: resources and a uniform interface; evolves by adding fields, or with versioned URLs/headers
JSON-RPC	None (by convention)	JSON	HTTP, WebSocket	Method names shared informally; no schema checks, so breakage shows up at run time
gRPC	Protocol Buffers	Binary (Protobuf)	HTTP/2	Tight contract via generated stubs; evolves safely through field numbers
GraphQL	GraphQL SDL	JSON	HTTP (one endpoint)	Clients select the fields they need; evolves by adding fields and deprecating old ones
Thrift	Thrift IDL	Binary, compact or JSON	TCP, HTTP	Generated stubs; field ids allow evolution as in Protobuf

Protocol Buffers and gRPC

- **Protocol Buffers:** Google's binary, schema-based serialization format, designed in 2001 as an alternative to XML and open-sourced in 2008.
- **gRPC:** uses .proto files as its IDL; the compiler generates client stubs and server skeletons in many languages, over HTTP/2.





https://medium.com/@akshitjain_74512/inter-service-communication-with-grpc-d815a561e3a1

gRPC communication patterns

```
service Chat {
  rpc Greet  (HelloRequest)   returns (HelloReply);          // unary
  rpc Watch  (Filter)         returns (stream Update);      // server streaming
  rpc Upload (stream Chunk)   returns (UploadSummary);      // client streaming
  rpc Talk   (stream Message) returns (stream Message);     // bidirectional
}
```

Pattern	Client sends	Server sends	Typical use
Unary	one request	one response	classic RPC: lookups, commands
Server streaming	one request	a stream of responses	subscriptions, large result sets
Client streaming	a stream of requests	one response	uploads, telemetry batches
Bidirectional	a stream	a stream (independent)	chat, interactive sessions

All four patterns run over a single HTTP/2 connection, with flow control per stream. Only the unary pattern corresponds to classic request-reply.

Conclusions

- Explain the request-reply protocols (R, RR, RRA) and the call semantics they support: maybe, at-least-once, at-most-once.
- Describe the RPC/RMI stub-skeleton model and why latency and partial failures limit transparency.
- Describe the role of an interface definition language and why language independence matters.
- Compare API styles (RPC, resource-oriented, query-oriented) and serialization formats (JSON vs. schema-based binary).

Exercise: creating a simple gRPC server and client

Server requirements

1. Define the service and a Greet method in a .proto file (gRPC IDL): Greet accepts a single string parameter and returns a greeting message.
2. Create a gRPC server in Python that offers this service.
3. Implement the server logic of Greet, building the greeting from the provided string.

Client 1 requirements

1. Create a Python client that connects to the server.
2. Call Greet with a custom name and print the greeting received.

Client 2 requirements

1. Create a second client that connects to the same server.
2. Call Greet with a different name and print the greeting received.

Extension: write Client 2 in another language (e.g. Java or Go) from the same .proto file, and set a deadline on the call to observe what the client sees when the server is slow or down.

References

Textbooks

- G. Coulouris, J. Dollimore, T. Kindberg, G. Blair. *Distributed Systems: Concepts and Design*, 5th ed. Addison-Wesley, 2012. Chapters 4–5.
- M. van Steen, A. S. Tanenbaum. *Distributed Systems*, 4th ed. distributed-systems.net, 2023. Chapter 4.
- W. Stallings. *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018. Chapter 5.

Papers

- A. D. Birrell, B. J. Nelson. Implementing Remote Procedure Calls. *ACM Transactions on Computer Systems* 2(1), 1984.
- J. Waldo, G. Wyant, A. Wollrath, S. Kendall. A Note on Distributed Computing. Sun Microsystems Laboratories, TR-94-29, 1994.
- R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, UC Irvine, 2000.

Specifications and documentation

- RFC 9110, HTTP Semantics (2022); RFC 9114, HTTP/3 (2022).
- gRPC documentation: grpc.io/docs. Protocol Buffers documentation: protobuf.dev.

Questions?

